What Does a Neural Simulation Tick Cost?

A Cost-Normalized Evaluation of a Sparse GPU Simulator

zxcAsD01
Independent researcher
https://github.com/zxcAsD01-d/Neurosim-GPU

October 2026

Abstract

Neural simulators are usually evaluated against other simulators, which mixes algorithmic differences with implementation quality. We argue for a narrower and more portable denominator: the cost of one sparse matrix multiplication over the same synapses, on the same device, with the same batch. We present a fixed-topology sparse simulator built around that accounting and report what the denominator reveals. On a laptop GPU, a complete neural tick—sparse reduction, tanh, leak, gain and clip—costs 0.74–0.82× a same-parameter `cuSPARSE` multiplication at batch one and 2.8–6.6% more than a matching custom SpMM; the fused kernel beats the library call only because it fuses work the library cannot. Persistent-current event propagation is 3.0–4.5× faster than the matrix path when 2.6% of edges emit, and on trained MNIST networks propagating one fifth of the edges changes accuracy by 0.1 percentage points. We also report where the approach does *not* pay: the reference event path is 5–81× slower than dense propagation on CPU, eligibility scanning destroys whole-tick event sparsity, and a dopamine-gated online plasticity rule produced no measurable behavioural change in our control task—an ablation shows the rule had no leverage on the synapses that mattered. We release the implementation, 105 experiment runs, raw payloads and a device-agnostic performance harness so that every number can be regenerated or contradicted.

1 Introduction

Simulation papers usually answer "how fast is your simulator?" by comparing against another simulator. That comparison is useful, but it confounds three different things: the cost of the sparse linear algebra the model requires, the cost of the neural update around it, and the engineering quality of the implementation. A simulator can win the comparison by being faster at sparse math, or lose it for reasons that have nothing to do with the model.

This paper takes the position that a neural simulator should first be measured against *the sparse operation it is made of*. For a fixed topology with M synapses, N neurons and batch B, one tick is

$$c_t = W\, s_t + d_t, \tag{1}$$

$$s_{t+1} = \mathrm{clamp}\big(\alpha \odot s_t + (1-\alpha) \odot g \odot \tanh(c_t),\, 0,\, 1\big), \tag{2}$$

with W sparse, $s_t \in [0,1]^{B\times N}$, and per-neuron leak α and gain g. The work is $O(BM + BN)$. The interesting question is not whether this is asymptotically a sparse matrix multiplication—it is—but how much the `tanh`, the leak, the clip, the Python dispatch and the memory traffic add on top of it in practice.

We built a simulator around this question and report four kinds of evidence. First, *tick parity*: the fused tick against a same-parameter sparse multiplication on the same device. Second, *event parity*: what happens when only state deltas above a threshold are propagated, both in time and in accuracy on trained networks. Third, *task evidence* on MNIST digits and an inverted pendulum, including negative results. Fourth, a *reproducibility artifact*: the implementation, the experiment grid, the raw payloads and the scripts that regenerate the tables in this paper.

Contributions.

1. A cost-normalized evaluation protocol for sparse neural simulation that separates simulator overhead from sparse-math cost, with two denominators (a library SpMM and a matching custom SpMM) and explicit controls (same topology, weights, batch, activity, and device state).

2. Measurements of a fused sparse tick from 1M to 50M synapses: 0.74–0.82× the `cuSPARSE` denominator at batch one, 1.15× at batch eight, and 2.8–6.6% over a matching custom SpMM.

3. A bounded-error characterization of emitted-event propagation on *trained* weights rather than random ones: propagating 21% of edges costs 0.11 accuracy points on MNIST, 8.4% costs 0.55.

4. A negative result with a diagnostic: dopamine-gated three-factor plasticity produced no behavioural change in a closed-loop control task, and an ablation shows why—zeroing every recurrent weight still left 193.8/200 survival steps, so the rule had no leverage.

5. An explicit account of where event sparsity stops paying: eligibility scanning is $O(BM)$ per tick, and the reference event scatter is slower than dense propagation on CPU.

We do not claim a new simulation algorithm. Sparse spiking simulation, event-driven propagation and three-factor plasticity are mature ideas; the contribution is measurement discipline, an integrated reference implementation, and a clear statement of what the numbers do and do not support.

2 Related Work

Brian 2 provides a mature, equation-oriented simulator with strong CPU and code-generation performance [10]; NEST targets large-scale distributed networks [5]; GeNN generates device code for GPU execution [11]. Reviews of the design space date back to Brette et al. [1]. Event-driven simulation of networks with delays and dynamic synapses has a long history [6], and the practical question of when it beats time-driven simulation is exactly the trade-off we quantify here.

On the learning side, three-factor plasticity rules that combine a local eligibility trace with a neuromodulatory signal are well established [4], and the dopamine reward-prediction-error interpretation is standard [9]. Libraries such as Norse [8] and snnTorch [3] make spiking layers usable inside autodiff frameworks, while neuromorphic hardware such as Loihi explores on-chip learning [2]. Our implementation uses PyTorch [7] for the reference paths and a small custom CUDA extension for the fused paths.

Relative to this literature, our claim is deliberately modest: we do not propose a better simulator, but a way to report what a simulator costs, and a set of measurements that other implementations can be compared against.

3 System

3.1 State and topology

The topology is a fixed directed multigraph with N neurons and M synapses after coalescing duplicate edges. Two views are materialized: a canonical source-major edge list (source, destination, weight) used by event propagation, and a destination-major CSR view used by the matrix path. A monotonic *weight version* lets a persistent-current runtime detect out-of-band weight changes and rebuild its cached current. Positions, chemical concentrations, eligibility and delay history each have a single owner and a documented lifetime; the ownership table is part of the repository documentation.

3.2 Runtimes

Four paths share the same state layout:

- **Matrix** (reference): one `torch.sparse.mm` per tick plus elementwise update.
- **Persistent current (PELS)**: keeps Ws^{emit} as state and propagates only emitted deltas; mathematically equivalent to the matrix path at threshold zero.
- **Native matrix**: one fused CUDA kernel over a single destination-major weight array.
- **Native event**: fused advance, compaction and source-major scatter.

The first two are portable and readable; the last two are the performance paths and are inference-only. Training uses the matrix path or a pointwise native adjoint, never the event kernel.

3.3 Emitted-event propagation

Each tick produces a state delta $\Delta s = s_{t+1} - s^{\text{emit}}$. Edges whose source delta is below a threshold θ are not traversed; instead the persistent current is updated by scattering only the retained deltas:

$$c \leftarrow c + W\,\mathbf{1}[|\Delta s| \geq \theta] \odot \Delta s. \tag{3}$$

With $\theta = 0$ this is the matrix path. With $\theta > 0$ the error is bounded per tick by the suppressed delta mass, but it compounds across ticks; the repository treats the threshold as a contract parameter with a validation protocol, not as a free knob.

3.4 Neurochemistry and schedule

A regional dopamine field supports tonic, burst and dip components, volume-weighted diffusion, uptake, transport delay, autoreceptor feedback and D1/D2 Hill occupancy. Plasticity is three-factor: pre/post traces form a per-edge eligibility, and a dopamine signal gates a sign-preserving weight update. These run at their own cadence—chemistry every K_c ticks, migration every K_m, structural change every K_s—so the neural tick stays $O(BM + BN)$ while the slow processes pay their cost only when they run.

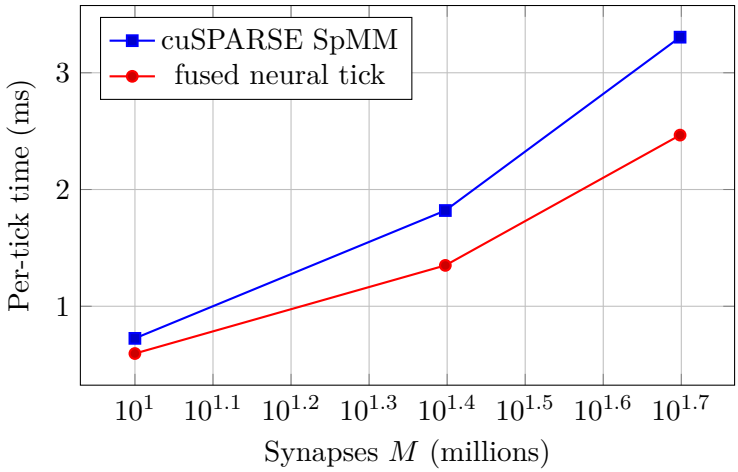


Figure 1: Fixed-topology tick cost against a same-parameter cuSPARSE multiplication (RTX 5070 Laptop, batch 1). The complete neural update is cheaper than the library denominator because it fuses the elementwise work.

4 Cost-Normalized Evaluation

4.1 Denominators

We report two ratios. The *library ratio* is $\rho_{\text{lib}} = T_{\text{tick}}/T_{\text{cuSPARSE}}$, where the denominator is a sparse multiplication with the same W, batch and dtype, measured on the same device in the same session. The *matching ratio* is $\rho_{\text{match}} = T_{\text{tick}}/T_{\text{SpMM}}$, where the denominator is our own destination-major SpMM kernel without the elementwise update.

The two answer different questions. ρ_{lib} is what a user gets by replacing a library call with the simulator; it can be below one because fusion removes intermediate traffic. ρ_{match} isolates the cost of the neural update itself. Reporting only one of them invites the wrong conclusion: $\rho_{\text{lib}} < 1$ does not mean the dynamics are free, and $\rho_{\text{match}} \approx 1$ does not mean a library swap is a win.

4.2 Metrics and controls

We report per-tick time (median after warmup), full-graph-equivalent throughput in synapses per second, bytes per synapse for the deployed layouts, and, for the event path, the fraction of edges that actually emit. Controls: identical topology object and weights across runtimes; identical initial state and drive; batch and dtype fixed; the device otherwise idle; topology construction and JIT compilation outside the timed region.

For accuracy claims we use a different harness entirely (Section 5.5), because a fast tick that computes the wrong thing is not a result.

5 Results

5.1 Tick parity

Table 1 and Figure 1 summarize the GPU measurements (RTX 5070 Laptop, 8 GiB, CUDA 13.1, FP32). At batch one the complete tick costs 0.821×, 0.742× and 0.746× the `cuSPARSE` denominator at 10M, 25M and 50M synapses. Against the matching custom SpMM the same ticks cost 1.066×, 1.028× and 0.918×: the neural update adds 2.8–6.6% over the sparse reduction, and at the largest

Table 1: GPU tick parity. "raw" is a same-parameter `cuSPARSE` multiplication, "canonical" our destination-major SpMM without the update, "fused" the complete tick. Batch one except the last row.

N	M	B	raw (ms)	canonical (ms)	fused (ms)	fused/raw
166k	10.00M	1	0.725	0.558	0.595	0.821
166k	24.99M	1	1.820	1.313	1.350	0.742
166k	49.95M	1	3.305	2.686	2.466	0.746
50k	1.00M	8	0.447	0.460	0.513	1.148

Table 2: Emitted-event threshold on MNIST, aggregated over 53 trained sparse-brain configurations. Accuracy is measured through the persistent-current runtime, so $\theta > 0$ is the bounded-error path; Δ is relative to the same run's matrix-path accuracy.

θ	Accuracy (%)	Δ (pp)	Active edges (%)
0.00	90.29 ± 2.79	+0.00	96.2
0.02	90.29 ± 2.84	-0.00	59.9
0.05	90.30 ± 2.72	+0.01	37.6
0.10	90.18 ± 2.75	-0.11	21.3
0.20	89.74 ± 2.55	-0.55	8.4

graph the fused kernel is slightly cheaper than the matching SpMM alone because the state update is folded into the reduction pass.

At batch eight the picture changes: 1.148× the library denominator and 1.115× the matching SpMM. The elementwise work now has more data per synapse to move, and the fused kernel loses the batch-one advantage. This is the expected shape—fusion helps when the arithmetic intensity is low—but it is worth stating because it bounds the claim: the parity result is a batch-one result.

5.2 Event sparsity

With $\theta = 0.05$ and 20 warmup ticks, 2.64% of edges emit. The native event kernel then takes 0.200 ms versus 0.595 ms for the matrix tick at 10M synapses, 0.301 versus 1.350 ms at 25M, and 0.123 versus 0.513 ms at 1M with batch eight: a 3.0–4.5× speedup.

The more interesting question is what this costs in accuracy. We trained 56 sparse-brain configurations on MNIST (varying width, degree, ticks, data and leak), then re-ran each through the persistent-current runtime at several thresholds. Table 2 and Figure 2 aggregate the result: 37.6% of edges active at $\theta = 0.05$ changes accuracy by +0.01 percentage points, 21.3% active at $\theta = 0.1$ costs 0.11 points, and 8.4% active at $\theta = 0.2$ costs 0.55 points.

Two caveats matter. First, this measures forward propagation only. With eligibility enabled, the dopamine runtime still scans all M edges per tick, so whole-tick event sparsity requires eligibility to be off and no weight update to occur on that tick. Second, the activity fraction depends on the dynamics and the warmup, not on θ alone: the same threshold produced 2.64% activity in the GPU benchmark and tens of percent on the MNIST runs. The relationship between active edges and accuracy is the transferable result, not the threshold value.

5.3 Memory

The mutable topology stores two endpoints, two weight views, two row-pointer arrays and a permutation: $24M + 8N$ bytes, or 3.0 GB at 125M synapses. A read-only inference view uses one weight array plus event indices, about $16M + 8N$ bytes, or 2.0 GB. Measured bytes per synapse range from

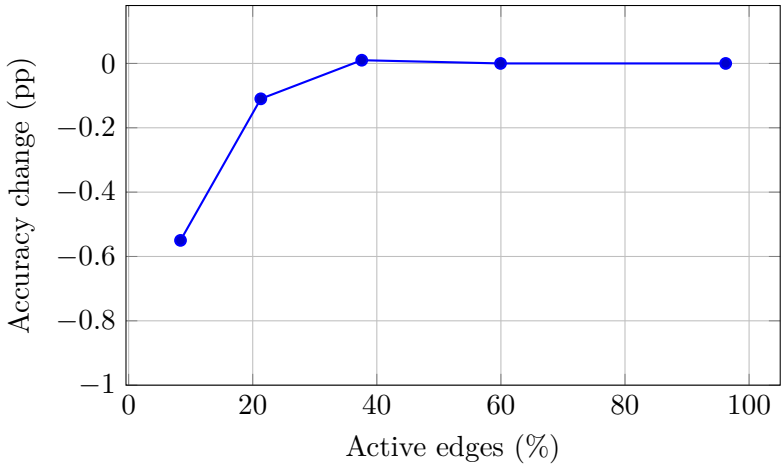


Figure 2: Bounded-error event propagation: accuracy change against the fraction of edges that emit, averaged over 56 trained sparse-brain configurations. Propagating one fifth of the edges costs about a tenth of a point.

Table 3: MNIST test accuracy (10,000 training samples, 10 epochs, 3 seeds; mean ± population standard deviation). "Params" counts trainable scalars; "synapses" counts coalesced edges of the fixed topology.

Model	Test accuracy (%)	Params	Synapses
Logistic regression	91.8 ± 0.3	7,850	–
MLP, $h = 6$	87.3 ± 0.9	4,780	–
MLP, $h = 64$	94.7 ± 0.0	50,890	–
Sparse brain (N=256, d=8, T=8)	91.1 ± 0.2	4,586	2,016
Dense RNN (N=256, T=8)	95.0 ± 0.3	269,066	–

24.2 to 25.0 for N between 2k and 32k, consistent with the layout. These constants are structural, not implementation accidents: removing a view or an index array changes the constant, not the scaling.

5.4 Training

Forward and backward for a same-parameter sparse RNN ($N = 16{,}384$, $M = 998{,}187$, $B = 4$, $T = 32$) take 185.6 ms. PyTorch matrix dynamics take 209.9 ms (1.131×); the native fused pointwise dynamics take 199.5 ms (1.075×, 93% same-parameter efficiency). The native analytic pointwise backward removes about half of the overhead (13.1% → 7.5%). Sparse-weight backward still uses `cuSPARSE`; a fused whole-rollout adjoint is not implemented, and we do not claim training is free: 7.5% is a cost, not a speedup.

5.5 Task evidence

All task experiments run on CPU, in pure PyTorch, and are *not* performance measurements; they exist to check that the stack learns and controls, and to expose failure modes. Table 3 and Figure 3 report MNIST accuracy at 10k training digits, 10 epochs, three seeds.

A sparse brain with 4,586 trainable scalars reaches 91.1%, against 87.3% for a parameter-matched MLP (4,780 scalars) and 91.8% for logistic regression (7,850). The fixed sparse recurrence is worth 6.3 points over the same architecture without recurrence (84.7%), so the gain is not merely a random projection. But the absolute accuracy is below a linear classifier and far below a wide

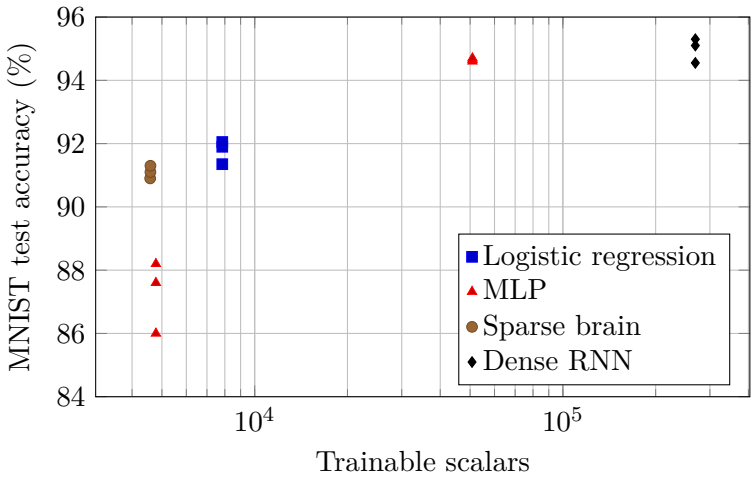


Figure 3: Parameter efficiency on MNIST (10k training digits, 3 seeds). The sparse brain sits at 4.6k trainable scalars between the parameter-matched MLP and the wide baselines; it does not beat a linear classifier.

Table 4: Cart-pole closed-loop evaluation (100 episodes, 3 seeds). Survival is out of 200 steps; success requires ≥ 195 steps. Angle RMS is in degrees. BC accuracy is the held-out imitation accuracy.

Policy	Protocol	Survival	Success (%)	Angle RMS	BC (%)
LQR	nominal	200.0	100	0.36	–
MLP (BC)	nominal	200.0	100	0.36	96.7
Sparse brain (BC)	nominal	200.0	100	0.36	88.7
LQR	hard	200.0	100	0.96	–
MLP (BC)	hard	200.0	100	0.97	96.2
Sparse brain (BC)	hard	200.0	100	1.12	85.0

MLP (94.7%) or a dense RNN with 59× the parameters (95.0%). Fixed topology buys parameter efficiency, not accuracy.

Under additive Gaussian input noise (Figure 4) the ordering reverses. At $\sigma = 0.3$ the sparse brain reaches 72.9%, against 55.1% for the wide MLP and 57.3% for logistic regression; at $\sigma = 0.2$ the gap is 84.4% versus 74.8%. The saturating dynamics appear to act as a bounded denoiser, while the ReLU baselines propagate noise to the readout. The advantage does not extend to occlusion, where the sparse brain is slightly worse, or to a two-pixel shift, where it is in the middle of the pack. Robustness is direction-dependent.

On the inverted pendulum (Table 4) both cloned policies sit at the 200-step ceiling, so imitation accuracy (96.7% versus 88.7%) does not translate into a closed-loop difference. Failures require extreme conditions (Table 5): action delay of six steps collapses both policies to about 30 steps, and noise $\sigma = 0.2$ degrades both. The failure direction differs—the sparse brain loses stability under delay four while the MLP remains at ceiling, and a four-fold pole length collapses the MLP (38.8 steps) while the sparse brain survives (198.7)—which suggests different frequency responses rather than a general robustness ranking.

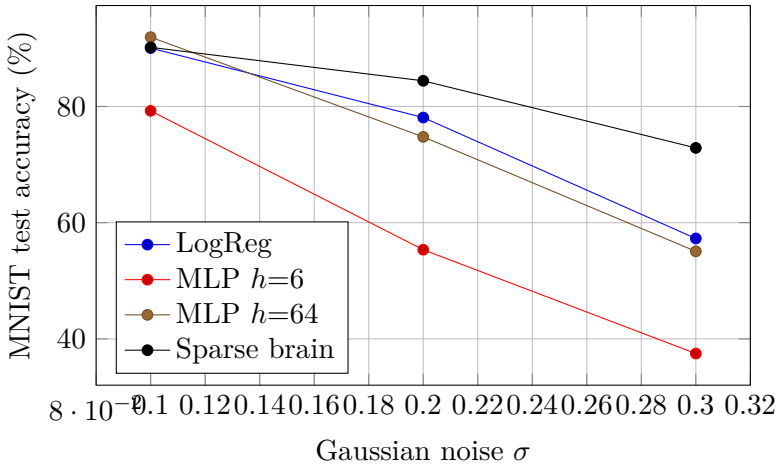


Figure 4: Accuracy under additive Gaussian input noise (3 seeds). The saturating sparse core degrades more slowly than the ReLU baselines at high noise.

Table 5: Robustness grid on the hard protocol (40 episodes per condition, 2 seeds): mean survival steps out of 200. Both policies sit at the ceiling for most conditions; the failures are informative.

Condition	Sparse brain	MLP
nominal	200.0	200.0
impulse	200.0	200.0
noise=0.1	197.3	200.0
noise=0.2	102.2	127.9
delay=1	200.0	200.0
delay=2	200.0	200.0
delay=4	162.8	200.0
delay=6	37.2	29.1
length_scale=0.5	200.0	200.0
length_scale=2	193.7	200.0
length_scale=4	198.7	38.8
mass_scale=0.3	200.0	200.0
mass_scale=3	200.0	200.0

5.6 A negative result, and why

We tested whether dopamine-gated three-factor plasticity could adapt a cloned policy after an unannounced plant change (actuator authority reduced from 6 N to 1 N at episode ten). Frozen and online variants were statistically indistinguishable: post-change survival 130.3 versus 129.1 steps over three seeds, with online slightly worse. The rule did move weights ($\|\Delta W\| = 0.80$, about 8% of the initial norm).

The ablation explains it. Zeroing every recurrent weight while keeping the sensory projection and readout leaves 193.8/200 survival steps on the hard nominal protocol (seeds: 200.0, 196.4, 184.9). The learned recurrence contributes almost nothing to behaviour; the policy is driven by the fixed feedforward path, so a rule that updates only recurrent weights has no lever. This is not evidence against the plasticity rule. It is evidence that the testbed, not the rule, was the limiting factor—and a reminder to run an ablation before claiming that an online rule "works".

5.7 The CPU reference path

The portable matrix runtime is also measured on CPU (16 threads, PyTorch 2.14.0+cpu, median of 30 ticks). The complete tick costs 1.18–1.82× a raw sparse multiplication at batch one and 1.82–3.02× at batch eight, with throughput from 0.15 to 2.9 G synapses/s. The Python event scatter, by contrast, is 5–81× *slower* than dense propagation. This is the sharpest boundary in the paper: event sparsity is a property of the fused kernel. A reference implementation that validates bounded error is not a performance path, and reporting it as one would be wrong.

6 Threats to Validity

One device. The GPU numbers come from one laptop GPU and one driver/toolkit combination. We report the stack with every number, and the harness is device-agnostic, but we have not reproduced them elsewhere.

No head-to-head comparison. We compare against sparse primitives, not against Brian 2, NEST, GeNN or Norse. A simulator-level comparison is the natural next experiment and is not claimed here.

Scale. The largest executed graph is 49.95M synapses. The 125M point is a projection from measured throughput, not a measurement; the reason is that random topology construction has a much larger temporary memory peak than loading a prebuilt graph.

Activity depends on the run. The event-path speedup is reported at 2.64% active edges, and the accuracy tables at 8–96% depending on θ and dynamics. Neither transfers to a different activity regime without re-measurement.

Biology. The neurochemical model is a functional proxy. Parameters are not calibrated to recordings, and no biological claim is made.

Task probes. Behaviour cloning is not reinforcement learning; the ceiling is the LQR expert. The MNIST runs use 10k training digits and three seeds without significance testing, so relative ordering, not absolute percentages, is the finding.

7 Reproducibility

The repository contains the implementation, the 105-run experiment grid, the raw JSON payloads, the aggregate tables and two harnesses: a device-agnostic performance suite and a task-level suite. Accuracy and survival metrics are seed-deterministic; latency is not. A CPU reference baseline is committed with its machine profile. Every table in this paper is generated by `paper/make_assets.py` from the committed results, so a disagreement can be traced to a payload rather than to prose. Documentation separates the source of record for measurements from the interpretation and from the acceptance criteria.

8 Conclusion

A sparse neural simulator should be able to say what one tick costs relative to the sparse multiplication it is built on. On our measurements that ratio is 0.74–0.82× a library SpMM at batch one and 2.8–6.6% over a matching custom SpMM: the neural update is cheap, and the remaining cost is memory movement and dispatch. Event propagation buys 3.0–4.5× at low activity and, on trained networks, costs about a tenth of an accuracy point at one fifth of the edges. The approach stops paying when eligibility scanning or a reference (non-fused) event path enters the picture, and an online plasticity rule with no leverage on behaviour is indistinguishable from no rule at all. Those boundaries are the useful part of the result.

References

[1] Romain Brette, Michelle Rudolph, Ted Carnevale, Michael Hines, David Beeman, James M. Bower, Markus Diesmann, David F. M. Goodman, Frederick C. Harris, Michael Zirpe, et al. Simulation of networks of spiking neurons: a review of tools and strategies. *Journal of Computational Neuroscience*, 23(3):349–398, 2007.

[2] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, et al. Loihi: a neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018.

[3] Jason K. Eshraghian, Max Ward, Emre O. Neftci, Xinxin Wang, Gregor Lenz, Garrick Dwivedi, Mohammed Bennamoun, Doo Seok Jeong, and Wei D. Lu. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 111(9):1016–1054, 2023.

[4] Nicolas Frémaux and Wulfram Gerstner. Neuromodulated spike-timing-dependent plasticity, and theory of three-factor learning rules. *Frontiers in Neural Circuits*, 9:85, 2016. doi: 10.3389/fncir.2015.00085.

[5] Marc-Oliver Gewaltig and Markus Diesmann. NEST (NEural Simulation Tool). *Scholarpedia*, 2(4):1430, 2007.

[6] Maurizio Mattia and Paolo Del Giudice. Efficient event-driven simulation of large networks of spiking neurons and dynamical synapses. *Neural Computation*, 12(10):2305–2329, 2000.

[7] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: an imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[8] Christian Pehle and Jens Egholm Pedersen. Norse: A library for spiking neural networks. `https://github.com/norse/norse`, 2021. Software library.

[9] Wolfram Schultz, Peter Dayan, and P. Read Montague. A neural substrate of prediction and reward. *Science*, 275(5306):1593–1599, 1997.

[10] Marcel Stimberg, Romain Brette, and Dan F. M. Goodman. Brian 2, an intuitive and efficient neural simulator. *eLife*, 8:e47314, 2019. doi: 10.7554/eLife.47314.

[11] Esin Yavuz, James Turner, and Thomas Nowotny. GeNN: a code generation framework for accelerated brain simulations. *Scientific Reports*, 6:18854, 2016. doi: 10.1038/srep18854.